1. Threat Hunting Methodologies

Proactive Threat Hunting

- Proactive threat hunting is a **security practice where analysts actively search for hidden threats** inside a network instead of waiting for security alerts.
- Traditional SOC monitoring relies on alerts from tools like **SIEM, IDS, or EDR**, but attackers often bypass these automated detections.

Hypothesis-Driven Hunting

- Hypothesis-driven hunting begins with a specific assumption about attacker activity in the environment.
- Analysts create a hypothesis based on known attacker techniques, threat intelligence, or unusual system behavior.
- For example, an analyst may assume that an attacker is abusing valid credentials **(MITRE ATT&CK T1078)**.

MITRE ATT&CK-Based Hunting

- Threat hunting is often guided by the MITRE ATT&CK framework, which documents real-world attacker techniques and tactics.
- Instead of randomly searching logs, analysts hunt for behaviors related to specific attack techniques such as phishing, credential abuse, or command execution.
- Each technique in MITRE ATT&CK has a unique identifier **(e.g., T1078 – Valid Accounts)**.

SqRR Threat Hunting Framework

- The **SqRR framework** is a structured method used to conduct threat hunting effectively.
- **Search:** Analysts identify suspicious patterns or anomalies in logs and system behavior.
- **Query:** Security tools such as **Splunk or Elastic** are used to run queries and filter relevant data.
- **Retrieve:** Investigators collect detailed evidence such as system logs, endpoint activity, or network traffic.

TaHiTI Threat Hunting Framework

- **TaHiTI (Targeted Hunting integrating Threat Intelligence)** focuses on using threat intelligence during threat hunting.
- Analysts collect indicators such as **malicious IP addresses, domains, file hashes, or attacker signatures** from threat intelligence feeds.
- If a match is found, analysts investigate the affected systems and potential compromise paths.
- This method helps security teams quickly identify known threats and adversary infrastructure inside their environment.

Data Sources Used in Threat Hunting

- Effective threat hunting requires collecting and analyzing **multiple types of security data**.
- **Endpoint Detection and Response (EDR) logs** provide details about process execution, file activity, and endpoint behavior.
- **Network traffic logs** reveal communication patterns and possible command-and-control (C2) connections.
- **Authentication logs** help detect suspicious login attempts, credential misuse, or privilege escalation.

Key Objectives of Threat Hunting

- The main objective is to identify threats before automated security systems detect them.
- Threat hunting improves an organization's overall security posture by reducing attacker dwell time.
- It helps analysts develop the ability to recognize abnormal system behavior and investigate it effectively

How to Learn Threat Hunting

- Study threat hunting methodologies through resources like SANS threat hunting research papers.
- Analyze real attack techniques using the MITRE ATT&CK framework to understand attacker behavior.
- Practice writing log queries in SIEM platforms like Splunk or Elastic to identify suspicious activities.
- Review threat hunting case studies involving advanced persistent threats (APT groups).

2. Advanced SOAR Automation

SOAR Overview

- SOAR stands for Security Orchestration, Automation, and Response, a system designed to automate and streamline SOC operations.
- It connects different security tools (SIEM, EDR, Threat Intel platforms) into a single workflow-driven system.
- This improves **speed, consistency, and efficiency** in incident response.

Orchestration

- Orchestration means connecting multiple security tools and making them work together.
- For example, when a SIEM detects a suspicious IP, SOAR can automatically query:
  - Threat intelligence platforms (VirusTotal)
  - Firewall logs
  - EDR tools
- This eliminates the need for analysts to manually switch between tools.

Automation

- Automation focuses on **eliminating repetitive and time-consuming SOC tasks**.
- Common automated tasks include:
  - Creating incident tickets
  - Enriching alerts with threat intelligence
  - Sending alerts to analysts
- For example, when a phishing email is detected, SOAR can automatically:
  - Extract URLs
  - Check them against threat intel
  - Flag them as malicious or safe
- This reduces analyst workload and minimizes human error.

Response

- Response is about **taking action automatically when a threat is confirmed**.
- Instead of waiting for human approval, SOAR can execute predefined actions such as:
  - Blocking malicious IPs on firewall
  - Isolating infected endpoints
  - Disabling compromised user accounts
- This significantly reduces **Mean Time to Respond (MTTR)**.

Playbook Development

- A playbook is a **step-by-step automated workflow** used to handle specific incidents.
- Each playbook defines:
  - What triggers the workflow
  - What actions to perform
  - What decisions to make
- Good playbooks are:
  - Repeatable
  - Scalable
  - Easy to update

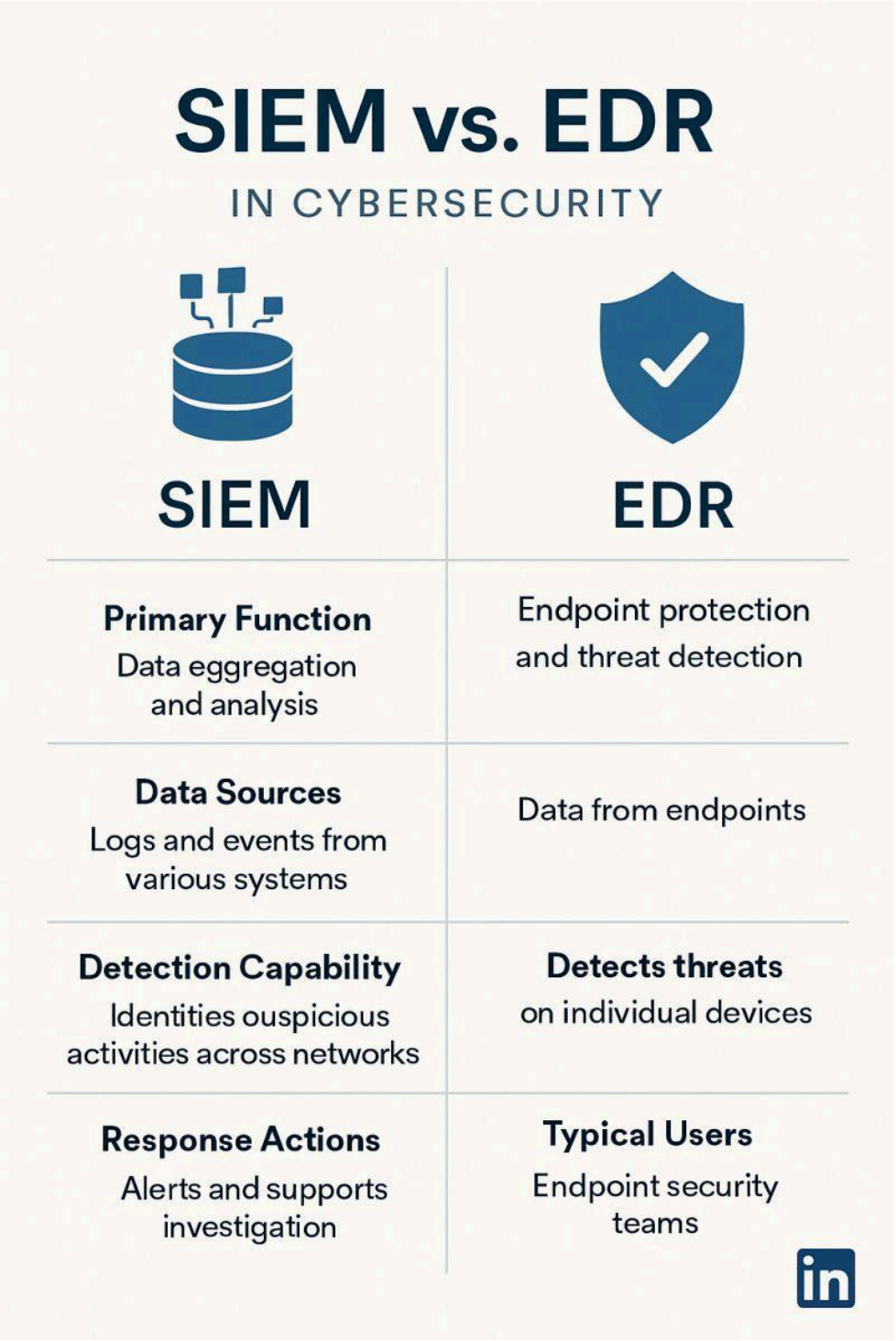
- Example: Phishing Playbook
  - **Detect email → Extract links → Check reputation → Block domain → Alert user**
- Playbooks are the **core of SOAR automation**.

Example Use Case

- A phishing email is reported by a user.
- SOAR automatically extracts:
  - Sender email
  - URLs
  - Attachments
- It checks these against threat intelligence sources.
- If malicious:
  - Blocks sender
  - Removes email from all inboxes
  - Creates incident ticket
- This entire process can happen in seconds instead of minutes/hours.

Integration with SIEM & EDR

- SOAR works closely with **SIEM and EDR tools**.
- SIEM generates alerts based on logs, while SOAR acts on those alerts automatically.
- EDR provides endpoint-level data, which SOAR can use to:
  - Investigate processes
  - Kill malicious processes
  - Isolate devices
- Example:
  - SIEM detects suspicious login → SOAR disables account via Active Directory
- This creates a fully automated detection + response pipeline.



Key Objectives of SOAR Automation

- Reduce manual workload of SOC analysts
- Improve incident response speed (MTTR)

- Ensure consistent and standardized response
- Minimize human errors in repetitive tasks
- Allow analysts to focus on complex investigations instead of basic tasks

---

How to Learn SOAR (Practical Approach)

- Study SOAR tools like Splunk SOAR (Phantom), TheHive, Cortex.
- Practice building simple playbooks (e.g., phishing detection workflow).
- Learn API integrations to connect tools like:
  - VirusTotal
  - Wazuh
  - Elastic
- Analyze real-world automation cases (e.g., phishing or malware response).
- Try creating your own mini-SOAR lab using open-source tools.

3. Post-Incident Analysis & Continuous Improvement

What is Post-Incident Analysis

- Post-incident analysis is the process of reviewing a security incident after it has been resolved.
- The goal is to understand what happened, how it happened, and how it can be prevented in the future.
- It ensures that incidents are not just closed, but used as learning opportunities.
- This phase is critical for improving SOC maturity and long-term security posture.

Root Cause Analysis (RCA)

- Root Cause Analysis is used to identify the actual reason behind an incident, not just the symptoms.
- It helps answer: *"Why did this incident happen?"*

Common RCA Techniques:

- 5 Whys Method:
  - Ask "why" repeatedly until the root cause is identified.
  - Example:
    - Why breach happened? → Phishing email clicked
    - Why clicked? → Lack of awareness
    - Why no awareness? → No training
- Fishbone Diagram (Ishikawa):

    - Breaks causes into categories like people, process, technology
- RCA helps fix core issues instead of temporary fixes

Lessons Learned Process

- After RCA, teams conduct a **"lessons learned" session**.
- This involves reviewing:
    - What worked well
    - What failed
    - What could be improved
- The output includes:
    - Improved detection rules
    - Better response procedures
    - Security awareness improvements
- This step ensures the SOC **evolves after every incident**.

Incident Documentation

- Every incident must be **properly documented** for future reference and compliance.
- Documentation includes:
    - Timeline of the attack
    - Indicators of compromise (IOCs)
    - Actions taken
    - Final resolution
- Proper documentation helps in:
    - Future investigations
    - Audit and compliance requirements
    - Knowledge sharing within the SOC team

SOC Metrics & KPIs

- Metrics are used to measure SOC performance and efficiency.

Important Metrics:

- **MTTD (Mean Time to Detect):**
    - Time taken to detect an incident
- **MTTR (Mean Time to Respond):**
    - Time taken to contain/respond
- **Dwell Time:**
    - Time attacker stays undetected
- **False Positive Rate:**
    - Number of incorrect alerts
- These metrics help identify weak areas in detection and response.

Continuous Improvement

- Continuous improvement means using insights from incidents to **enhance security systems and processes**.
- Example improvements:

    - Updating SIEM rules
    - Improving playbooks in SOAR
    - Strengthening firewall policies
- It ensures that the SOC becomes **stronger after every attack**.

Feedback Loop into Security Systems

- Findings from incidents are fed back into:
    - SIEM (better detection rules)
    - EDR (improved monitoring)
    - SOAR (optimized playbooks)
- This creates a **closed feedback loop**, where each incident improves future detection and response.

Real-World Example

- A phishing attack leads to credential theft.
- Post-incident analysis reveals:
    - Weak email filtering
    - No multi-factor authentication (MFA)
- Improvements made:
    - Stronger email filters
    - MFA implementation
    - Employee training
- Result: Future phishing attempts are **less effective**.

4. Adversary Emulation Techniques

What is Adversary Emulation

- Adversary emulation is the process of simulating real attacker behavior inside a controlled environment.
- Instead of random testing, you mimic actual tactics, techniques, and procedures (TTPs) used by real threat actors.
- The goal is to test whether your SOC can detect, respond, and defend against realistic attacks.

Purpose of Adversary Emulation

- The main purpose is to evaluate and improve detection and response capabilities.
- It helps identify:
    - Detection gaps in SIEM/EDR
    - Weak response processes

    - Missing security controls
- It ensures that your SOC is prepared for **advanced and targeted attacks (APT-level threats)**.

MITRE ATT&CK in Emulation

- Adversary emulation is heavily based on the MITRE ATT&CK framework.
- Instead of random attacks, you simulate specific techniques like:
    - T1566 → Phishing
    - T1210 → Exploitation of Remote Services
- This ensures the emulation is aligned with real attacker strategies.
- It also helps in mapping detections to specific ATT&CK techniques.

Emulation vs Penetration Testing

- **Penetration Testing:**
    - Focuses on finding vulnerabilities
    - Goal is to exploit systems
- **Adversary Emulation:**
    - Focuses on mimicking attacker behavior
    - Goal is to test detection and response

Adversary Emulation Workflow

- Define the attacker profile (e.g., APT28, insider threat)
- Select relevant MITRE ATT&CK techniques
- Simulate attack steps such as:
    - Initial access (phishing)
    - Execution (malicious scripts)
    - Persistence (backdoors)

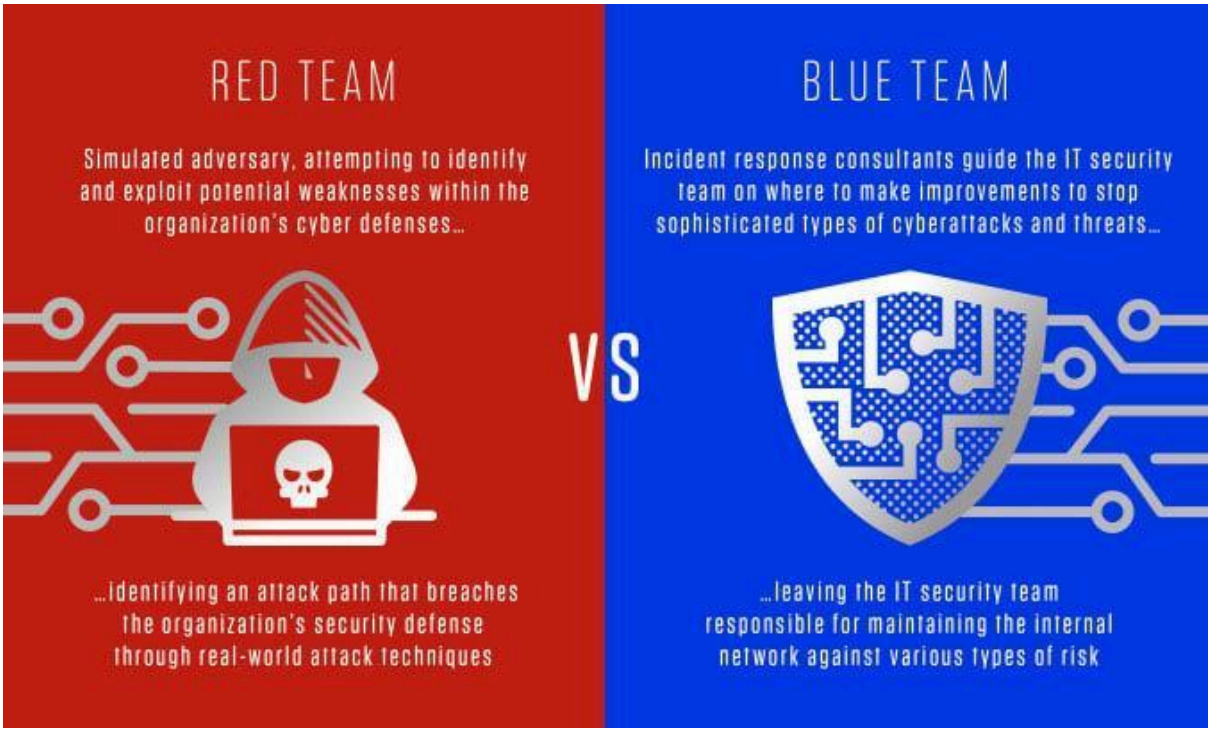
Tools for Adversary Emulation

- **MITRE Caldera:**
    - Automated adversary emulation platform
    - Simulates attacks based on ATT&CK techniques
- **Atomic Red Team:**
    - Provides small, testable attack simulations
- **Red Team tools (e.g., Metasploit, Cobalt Strike)**
    - Used for advanced attack simulations
- These tools help create realistic attack scenarios in lab environments.

Red Team vs Blue Team Collaboration

- **Red Team:** Simulates attackers
- **Blue Team:** Defends and detects attacks

- Adversary emulation connects both teams by:
  - Testing detection capabilities
  - Improving response strategies
- This collaboration helps build a **strong and adaptive security posture**.



Example Scenario

- Simulate a phishing attack (T1566)
- User clicks malicious link → malware executes
- Attacker gains access and tries lateral movement
- SOC monitors:
  - Did SIEM detect the login anomaly?
  - Did EDR flag the malware?
- If not detected → detection rules need improvement